



Algorithmics & C-Programming

Chapter 01: Introduction & History of the C-language

For the engineering school: ECE-Paris



Metadata

Edition:

1.0

 (2025.10.09)

Designed for the open standard:

C23

 (2024)

Written by: Simon Céleste VANDAMME

svandamme@omnesintervenant.com



01. FOREWORD

A. General Information

Module 1 - Foundations of Programming

TPE-001 Introduction & A Bit of History About the C Language

Discover where the C language comes from, why it was created, and how it shaped modern programming.

Learn about its legacy in UNIX, operating systems, and beyond.

TPE-001 Core Insight

- Understand the historical origins and purpose of the C language
- Recognize the inheritance and impact of C in modern programming
- Learn how C influenced operating systems, software...
- Start to get a glimpse of the C syntax
- Introduce basic algorithmic thinking

This lesson should take about: **1h**
to be completed.

B. Prerequisite Chapters

These are the recommended chapters to read:

- **NONE!**
This chapter doesn't require prior knowledge in C.





C. AI: Recommendation and Downsides

Artificial Intelligence (AI) tools show remarkable results, especially in the field of programming. Their integration into most Integrated Development Environments (IDEs) nowadays makes them more accessible than ever.

Used wisely, AI can be a valuable support in your journey to learn C programming. However, it can also be tempting to rely on it to complete entire projects or exercises in seconds.

When learning a new skill in any subject, remember this golden rule:

UNDERSTANDING > REWRITING PRE-DONE CONTENT

Note that I am NOT AGAINST the use of AI in your works.

However, I would highly appreciate it if you were transparent upon using it in your code and specify EVERY TIME why you are relying on it and how the snippet code works.

RECOMMENDATIONS

01. First think about the subject YOURSELF, write some elements down.
Then, use AI to continue your work if needed.

Your learning suffers when you let AI solve problems before trying yourself.
Skip the thinking, and you only train dependency... not skill.

As future engineers, you are expected to understand problems deeply and think creatively.
Keep training those skills: they are the foundation of your value as an engineer.

02. Use AI tools as personal teachers.
They are great to highlight your mistakes or improve your code quality.

Once done with a working program, you can prompt the AI on what is to be improved, for example:

- How to secure your code against errors?
- Ask for alternative ways to solve the problem. Did you use the most interesting way?
- How to improve your memory management or time execution?

03. YOU are responsible for the code you submit... not the AI.

Whether at school or in your future job, your program is your creation. You must fully understand it and be able to explain every part of it, even if you received significant help from AI.

AI has limits: it struggles with complex projects, can produce insecure code, and lacks real creativity.
As an engineer, it is your job to go beyond these limits.

02. TABLE OF CONTENTS

01. FOREWORD	2
A. General Information	2
B. Prerequisite Chapters.....	2
C. AI: Recommendation and Downsides	4
02. TABLE OF CONTENTS	5
03.SYLLABUS.....	6
03. TECHNICAL SPECIFICATION SHEET	14
A. REFERENCES	14
B. DOCUMENT SPECIFICATIONS	15



03. INTRODUCTION

A. Introduction to Algorithmics

“An algorithm is a **set of rules** that precisely defines a sequence of operations.”

Wikipedia

Algorithms are mostly used in the mathematics and computer science fields, but with a bit of creativity, you could use it anywhere, for example in:

- **Chemical classes:** to create proper instructions for an experiment beforehand.
- **In project management:** on how to achieve a complex goal with multiple actors involved in.
- ...

Overall, creating an algorithm is more about a way of thinking on how to solve logically a set of problems.

> Now, what does it have to do with programming?

Any programs that have ever been written so far can be translated into an algorithm. We could even say that programming languages are algorithms themselves.

You'll see when using and programming but writing in C requires strict instructions that describe each one a step for the computer to do.

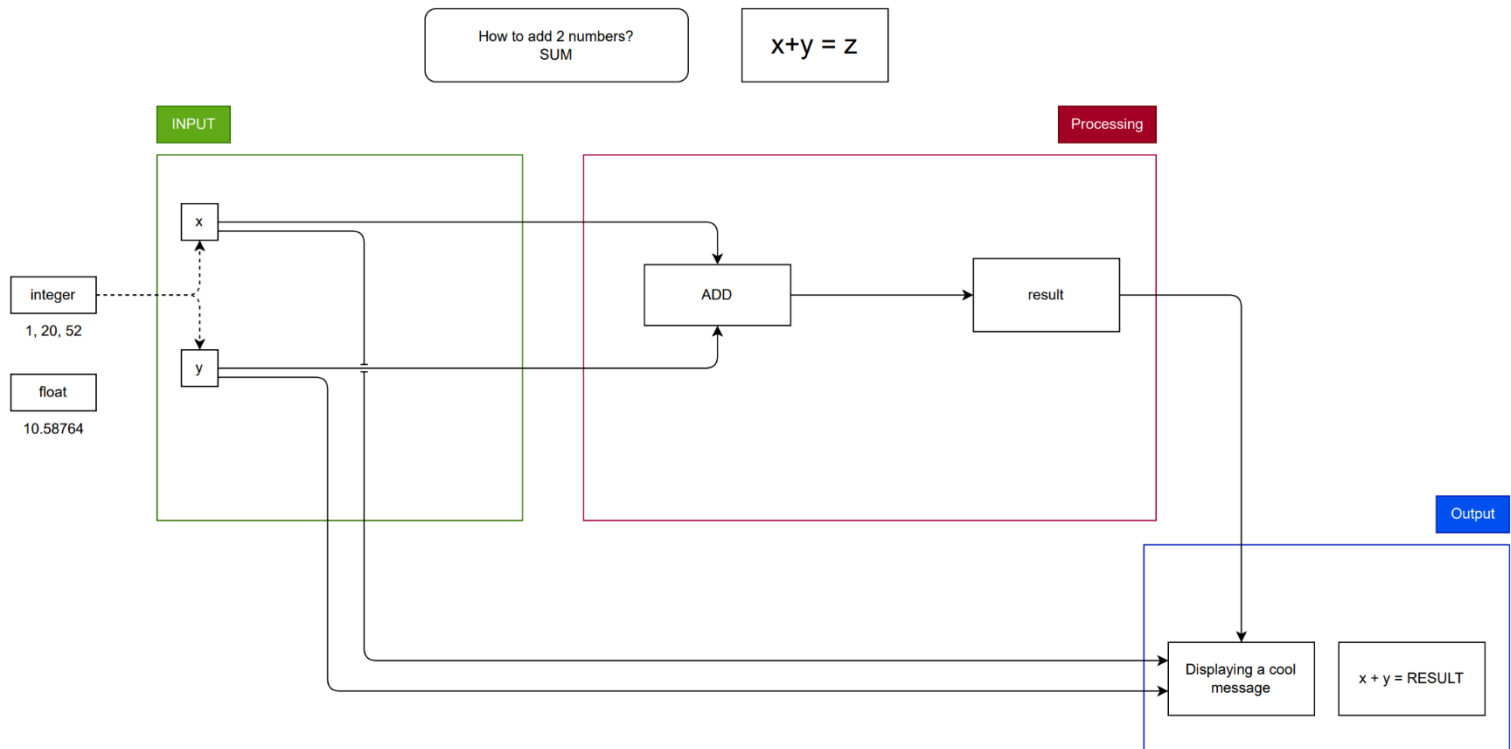
How to create an algorithm?

There are many ways to create one, but in most cases, it can be:

- **Visual:** with flowchart/diagram.
- **Pseudo-code:** a simplified programming-language.

Let's look at an example.

Let's suppose you are working for a calculator company, and are tasked to do the SUM of 2 numbers algorithm.



- **Conceptual steps:** input → process → output.
- **Simple examples** (e.g., calculating the average, converting Celsius to Fahrenheit).
- How algorithmic thinking will guide learning C.

B.What is Programming?

- Definition of programming and software development.

Computers themselves can't "think" or understand natural language — they only follow exact sequences of operations (instructions).

Definition:

Programming is the act of designing, writing, testing, and maintaining a set of instructions (called a program) that tells a computer what to do.

These instructions are written in a programming language (like C, Python, or Java) that acts as a bridge between human logic and machine execution.



- How programming solves real-world problems.
- High-level vs low-level languages (overview).



04. FOCUS ON THE C-LANGUAGE

A. The Birth of C

At **Bell Labs** (part of AT&T in the U.S.), **Dennis Ritchie** and **Ken Thompson** were working on the **UNIX operating system**, which was initially written in **assembly**. This made UNIX powerful but hard to maintain and adapt to new machines.

➤ **C = B improved**

- Data types (like int, char, etc.),
- Structures and arrays,
- Pointer arithmetic for memory access,
- Syntax suitable for system-level programming.

➤ **Goals of C**

1. Efficiency

C was designed to produce fast, compact machine code.

It lets programmers manipulate memory directly using pointers, manage performance-critical tasks, and write code that compiles almost as efficiently as assembly — but with much less effort.

- Example: operating system kernels, device drivers, and compilers rely on C's performance.

2. Portability

Before C, programs were tied to specific hardware architectures.

C, on the other hand, was designed to be “**portable assembly**.”

By compiling C code using a different compiler on another machine, developers could reuse the same source code almost unchanged.

3. System Programming

C was built primarily for **system-level programming**, not application development at first.

That means it was intended to:

- Write **operating systems** and **compilers**,
- Manage **hardware resources** (memory, files, input/output),
- Implement low-level logic efficiently.



B.C & UNIX

- How C was closely tied to the development of UNIX.
 - Portability and why it mattered for operating systems.
- Legacy: why many modern OS kernels and compilers still use C.
- Original UNIX (1973)
 - **Linux kernel** (1991): almost entirely in C, with a small amount of assembly.
 - **macOS, iOS, and Other Apple OS:** Darwin > BSD UNIX > C
 - **Windows:** Windows NT kernel > C & C++
System DLLs (like kernel32.dll, ntoskrnl.exe)

C. Influence of C on Modern Programming

- Languages inspired by C: C++, Objective-C, Java, C#, Go, Rust.
- How C shaped coding standards, memory management concepts, and procedural programming.

D. Characteristics of the C Language

- Procedural, compiled, low-level access to memory.

Procedural programming is a programming paradigm based on the concept of procedures (functions) — reusable blocks of code that perform specific tasks, executed in a defined order.

It's called procedural because it focuses on the procedure — how something is done — rather than just the result.

- **Sequential execution:** Code runs in order, top to bottom.



- **Use of functions/procedures:** Tasks are divided into logical, reusable sections.
- **Variables and data:** Data is stored in variables that can be modified and passed between procedures.
- **Control structures:** if, for, while, and switch guide the flow of execution.
- **Global and local scope:** Variables can exist for the entire program or just within a procedure.

- Performance, portability, and simplicity.
- Overview of C's building blocks (variables, functions, control structures) - without details yet.

E. Real-World Examples & Fun Facts

- C in embedded systems, gaming, operating systems, and scientific computing.
- Famous software written in C (UNIX, Linux kernel, Python interpreter).

F. Quick Demo

- Show a very simple "Hello World" program.
- Explain what it does at a high level (without going into syntax yet).



05. INTRODUCTION TO C

[CHEAT SHEET]

- ❓ C is old but foundational.
- ❓ Understanding C gives insight into how computers work “under the hood.”
- ❓ Algorithmic thinking is a key skill for programming.

06. EXERCISES

- 🔗 Timeline exercise: place C and its predecessors on a timeline.
- 🔗 Brainstorm: simple everyday tasks as algorithms.
- 🔗 Mini-pseudocode exercise: describe how to make a sandwich or calculate an average.



07. TECHNICAL SPECIFICATION SHEET

A. References

The following references and resources were used as guidance and inspiration in the development of the courses.

Books:

- “Apprenez à programmer en C”, 2012, Mathieu NEBRA
- “Algorithms for Dummies”, 2024, John Paul Mueller and Luca MASSARON

Websites / Online documentation:

- WordReference: <https://www.wordreference.com/> *Translation: French <> English*

Software / Tools:

- **IDE (C programming):** *Used for screenshots, programming and testing.*
 - CLion (*JetBrains*)
 - Code::Blocks
 - Visual Studio Code (*Microsoft*)
- **Artificial Intelligence:** *Help in designing the syllabus and create original exercises.*
 - ChatGPT (*OpenAI*)
 - Gemini (*Google*)
- **Writing/Editing software:** *Creating my courses, exercises, presentations...*
 - PowerPoint (*Microsoft*)
 - Word (*Microsoft*)



B. Document specifications

TECHNICAL INFORMATION	
Document Version	1.0
Release Date (YYYY.MM.DD)	2025.10.09
Course Title	ALGORITHMICS & C-PROGRAMMING
Chapter Name	Chapter 01 - Introduction & History of the C-language
Document ID	C-SYLLABUS-001
Keywords	C, IDE, program, worksheet, course materials, digital
File Name	C-Syllabus-v1.0.pdf
Page Count	15 pages
Contributors	▪ Nathalie AMSALLEM: Proofreader ▪ Léo BELUITE: Proofreader ▪ Drazic MARTIN: Proofreader and program tester ▪ Simon Céleste VANDAMME: Author
Maintainer Contact	celeste.vandamme@outlook.com
Target Audience	Designed for engineering students starting to learn the C-programming language.
Languages Available	▪ English: available ▪ French: not yet available
Answer Sheet	Not required (no exercises)
Confidentiality Level	Public
License	Creative Commons Attribution 4.0 International (CC BY 4.0) Link: https://creativecommons.org/licenses/by/4.0/



VERSION HISTORY			
Version	Release Date (YYYY.MM.DD)	Authors	Description / What is new in the version?
1.0	2025.10.09	- VANDAMME S.	Official release of the document.
1.1	2025.10.09	- VANDAMME S.	Fixed blue highlight on module + fix on a wrong course ID number